

PIZZA SALES ANALYSIS

A MySQL Query Portfolio · 13 Business Questions Solved

BASIC • **INTERMEDIATE** • **ADVANCED**

Vikram Kaushik

Dataset: Pizza Place Sales | Tool: MySQL Workbench



01

Basic Queries

Foundational aggregations to understand the scale of the business

BASIC · Q1

Retrieve the total number of orders placed.

KEY INSIGHT

The dataset covers 21,350 orders, giving a solid, high-volume base for reliable trend and revenue analysis across the full order history.

```
1  -- Retrieve the total number of orders placed.  
2  
3  Select Count(Order_id) From orders;
```



A screenshot of a database query result grid. The grid has a single column header 'Count(Order_id)' and a single row with the value '21350'. Above the grid, there are tabs for 'Result Grid', 'Filter Rows', 'Exports', and 'Wrap Cell Contents'.

Count(Order_id)
21350

BASIC · Q2

Calculate the total revenue generated from pizza sales.

KEY INSIGHT

Total revenue across all orders comes to \$817,860.05, calculated by joining order line items with pizza prices and summing quantity x price.

```
1  -- Calculate the total revenue generated from pizza sales.
2
3 • SELECT
4  ROUND(SUM(orders_details.QUANTITY * pizzas.price),
5         2) AS Total_Revenue
6  FROM
7      orders_details
8  JOIN
9      pizzas ON orders_details.PIZZA_ID = pizzas.pizza_id;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
Total_Revenue			
817860.05			

BASIC · Q3

Identify the highest-priced pizza.

KEY INSIGHT

The Greek Pizza is the most expensive item on the menu at \$35.95, making it a natural candidate for premium-tier promotions.

```
1  -- Identify the highest-priced pizza.
2
3 • SELECT
4     py.name, p.price
5 FROM
6     pizza_types py
7     JOIN
8     pizzas p ON py.pizza_type_id = p.pizza_type_id
9 ORDER BY p.price DESC
10 LIMIT 1;
11
```

Result Grid			Filter Rows:	Export: 	Wrap Cell Content: 	Fetch rows: 
	name	price				
▶	The Greek Pizza	35.95				

BASIC · Q4

Identify the most common pizza size ordered.

KEY INSIGHT

Large (L) is the clear customer favorite with 18,956 units ordered, ahead of Medium (15,635) and Small (14,403). XL and XXL sizes are niche.

```
1  -- Identify the most common pizza size ordered.
2
3  • SELECT
4      pizzas.size,
5      Sum(orders_details.QUANTITY) AS order_count
6  FROM
7      pizzas
8      JOIN
9      orders_details ON pizzas.pizza_id = orders_details.PIZZA_ID
10 GROUP BY pizzas.size
11 ORDER BY order_count DESC;
12
13
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
size	order_count			
L	18956			
M	15635			
S	14403			
XL	552			
XXL	28			

BASIC · Q5

List the top 5 most ordered pizza types along with their quantities.

KEY INSIGHT

The Classic Deluxe Pizza leads with 2,453 orders, closely trailed by Barbecue Chicken, Hawaiian, Pepperoni, and Thai Chicken — all within ~80 units of each other.

```
1  -- List the top 5 most ordered pizza types along with their quantities.
2 • SELECT
3     pizza_types.name,
4     SUM(orders_details.QUANTITY) AS order_count
5  FROM
6     orders_details
7     JOIN
8     pizzas ON orders_details.PIZZA_ID = pizzas.pizza_id
9     JOIN
10    pizza_types ON pizzas.pizza_type_id = pizza_types.pizza_type_id
11 GROUP BY pizza_types.name
12 ORDER BY order_count DESC
13 LIMIT 5;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
	name	order_count			
▶	The Classic Deluxe Pizza	2453			
	The Barbecue Chicken Pizza	2432			
	The Hawaiian Pizza	2422			
	The Pepperoni Pizza	2418			
	The Thai Chicken Pizza	2371			



02

Intermediate Queries

Multi-table joins and grouped analysis to reveal category and time patterns

INTERMEDIATE · Q6

Join the necessary tables to find the total quantity of each pizza category ordered.

KEY INSIGHT

Classic pizzas lead category volume with 14,888 units, followed by Supreme (11,987), Veggie (11,649), and Chicken (11,050) — a fairly balanced spread.

```
1 -- Join the necessary tables to find the total quantity of each pizza category ordered.  
2  
3 • Select pizza_types.category, sum(orders_details.QUANTITY) as quantity  
4 from orders_details join pizzas on orders_details.PIZZA_ID = pizzas.pizza_id  
5 join pizza_types on pizzas.pizza_type_id = pizza_types.pizza_type_id  
6 group by pizza_types.category  
7 order by quantity desc;
```



category	quantity
Classic	14888
Supreme	11987
Veggie	11649
Chicken	11050

INTERMEDIATE · Q7

Determine the distribution of orders by hour of the day.

KEY INSIGHT

Orders peak sharply at 12 PM (6,776 pizzas) and again at 1 PM and 6 PM, pointing to clear lunch and dinner rushes worth staffing around.

```
1  -- Determine the distribution of orders by hour of the day.
2 • SELECT
3      HOUR(orders.order_time) AS dayhour,
4      SUM(orders_details.QUANTITY) AS quantity
5  FROM
6      orders_details
7      JOIN
8      orders ON orders_details.ORDER_ID = orders.ORDER_ID
9  GROUP BY dayhour
10 ORDER BY quantity DESC;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
dayhour	quantity			
12	6776			
13	6413			
18	5417			
17	5211			
19	4406			
16	4239			
14	3613			
20	3534			
15	3216			
11	2728			
21	2545			
22	1386			
23	68			
10	18			
9	4			

INTERMEDIATE · Q8

Join relevant tables to find the category-wise distribution of pizzas.

KEY INSIGHT

The menu itself is well balanced across categories: Veggie and Supreme offer 9 pizza types each, Classic 8, and Chicken 6 — variety is fairly even.

```
1  -- Join relevant tables to find the category-wise distribution of pizzas.
2 • Select category, count(name)
3  from pizza_types
4  group by category;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
category	count(name)			
Chicken	6			
Classic	8			
Supreme	9			
Veggie	9			

INTERMEDIATE · Q9

Group the orders by date and calculate the average number of pizzas ordered per day.

KEY INSIGHT

On average, about 138 pizzas are sold per day, calculated using a subquery that first totals daily quantity before averaging across all order dates.

```
1  -- Group the orders by date and calculate the average number of pizzas ordered per day.
2  * SELECT
3      ROUND(AVG(quan)) AS Avg_order_per_Day
4  FROM
5      (SELECT
6          orders.ORDER_DATE, SUM(orders_details.QUANTITY) AS quan
7      FROM
8          orders_details
9      JOIN orders ON orders.ORDER_ID = orders_details.ORDER_ID
10     GROUP BY orders.ORDER_DATE) AS order_per_day;
11
```



The screenshot shows a database query result grid. The top bar includes tabs for 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. Below the tabs, there is a table with one column labeled 'Avg_order_per_Day' and one row containing the value '138'.

Avg_order_per_Day
138

INTERMEDIATE · Q10

Determine the top 3 most ordered pizza types based on revenue.

KEY INSIGHT

By revenue, Thai Chicken (\$43,434), Barbecue Chicken (\$42,768), and California Chicken (\$41,410) top the list — none of which are the top sellers by quantity, showing higher price points drive more revenue per order.

```
1  -- Determine the top 3 most ordered pizza types based on revenue.
2  • SELECT
3      pizza_types.name,
4      ROUND(SUM(orders_details.QUANTITY * pizzas.price),
5             0) AS revenue
6  FROM
7      pizzas
8      JOIN
9      orders_details ON pizzas.pizza_id = orders_details.PIZZA_ID
10     JOIN
11     pizza_types ON pizza_types.pizza_type_id = pizzas.pizza_type_id
12 GROUP BY pizza_types.name
13 ORDER BY revenue DESC
14 LIMIT 3;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
	name	revenue				
▶	The Thai Chicken Pizza	43434				
	The Barbecue Chicken Pizza	42768				
	The California Chicken Pizza	41410				

03

Advanced Queries

Window functions and CTEs to uncover revenue share, trends, and category leaders

ADVANCED · Q11

Calculate the percentage contribution of each pizza type to total revenue.

KEY INSIGHT

Using a CTE to hold total revenue, Classic contributes 26.91% of all sales, followed closely by Supreme (25.46%), Chicken (23.96%), and Veggie (23.68%) — no single category dominates.

```
1  -- Calculate the percentage contribution of each pizza type to total revenue.
2  • With total as ( SELECT
3      SUM(orders_details.QUANTITY * pizzas.price) AS Total_Revenue
4  FROM orders_details JOIN
5      pizzas ON orders_details.PIZZA_ID = pizzas.pizza_id
6  )
7  SELECT
8      pizza_types.category,
9      Round((SUM(orders_details.QUANTITY * pizzas.price) / total.Total_Revenue ) * 100,2) as revenue
10 FROM
11     pizza_types
12     JOIN
13     pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
14     JOIN
15     orders_details ON orders_details.PIZZA_ID = pizzas.pizza_id
16     CROSS JOIN
17     total
18 GROUP BY pizza_types.category, total.Total_Revenue
19 ORDER BY revenue DESC;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
category	revenue			
Classic	26.91			
Supreme	25.46			
Chicken	23.96			
Veggie	23.68			

ADVANCED · Q12

Analyze the cumulative revenue generated over time.

KEY INSIGHT

A running total built with SUM() OVER (ORDER BY order_date) shows steady, consistent daily growth — revenue crosses \$52,700 within the first 23 days of the year with no major dips.

```
1  -- Analyze the cumulative revenue generated over time.
2  • Select ORDER_DATE, Round(sum(revenue) Over(order by order_date),2) as cum_revenue
3  From(
4  Select orders.ORDER_DATE,
5  sum(orders_details.QUANTITY * pizzas.price) as revenue
6  from pizzas join orders_details on pizzas.pizza_id = orders_details.PIZZA_ID
7  join orders on orders.ORDER_ID = orders_details.ORDER_ID
8  group by orders.ORDER_DATE ) as daily_revenue
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	ORDER_DATE	cum_revenue			
▶	2015-01-01	2713.85			
	2015-01-02	5445.75			
	2015-01-03	8108.15			
	2015-01-04	9863.6			
	2015-01-05	11929.55			
	2015-01-06	14358.5			
	2015-01-07	16560.7			
	2015-01-08	19399.05			
	2015-01-09	21526.4			
	2015-01-10	23990.35			
	2015-01-11	25862.65			
	2015-01-12	27781.7			
	2015-01-13	29831.3			
	2015-01-14	32358.7			
	2015-01-15	34343.5			
	2015-01-16	36937.65			
	2015-01-17	39001.75			
	2015-01-18	40978.6			
	2015-01-19	43365.75			
	2015-01-20	45763.65			
	2015-01-21	47804.2			
	2015-01-22	50300.9			
	2015-01-23	52724.6			

ADVANCED · Q13

Determine the top 3 most ordered pizza types based on revenue for each pizza category.

KEY INSIGHT

A CTE plus `DENSE_RANK() OVER (PARTITION BY category ORDER BY revenue DESC)` ranks pizzas within their own category — surfacing each category's top 3 earners side by side, e.g. Thai Chicken leads Chicken, Classic Deluxe leads Classic.

```
1 -- Determine the top 3 most ordered pizza types based on revenue for each pizza category.
2 WITH PizzaRevenue AS (
3     SELECT
4         pizza_types.category,
5         pizza_types.name,
6         SUM(orders_details.QUANTITY * pizzas.price) AS Revenue
7     FROM pizzas
8     JOIN orders_details
9         ON pizzas.pizza_id = orders_details.PIZZA_ID
10    JOIN pizza_types
11        ON pizza_types.pizza_type_id = pizzas.pizza_type_id
12    GROUP BY pizza_types.category, pizza_types.name
13 ),
14 RankedPizza AS (
15     SELECT
16         category,
17         name,
18         Revenue,
19         DENSE_RANK() OVER (PARTITION BY category ORDER BY Revenue DESC) AS drnk
20     FROM PizzaRevenue
21 )
22 SELECT
23     category,
24     name,
25     Revenue,
26     drnk
27 FROM RankedPizza
28 WHERE drnk <= 3;
```

	category	name	Revenue	drnk
▶	Chicken	The Thai Chicken Pizza	43434	1
	Chicken	The Barbecue Chicken Pizza	42768	2
	Chicken	The California Chicken Pizza	41410	3
	Classic	The Classic Deluxe Pizza	38180	1
	Classic	The Hawaiian Pizza	32273	2
	Classic	The Pepperoni Pizza	30162	3
	Supreme	The Spicy Italian Pizza	34831	1
	Supreme	The Italian Supreme Pizza	33477	2
	Supreme	The Sicilian Pizza	30940	3
	Veggie	The Four Cheese Pizza	32266	1
	Veggie	The Mexicana Pizza	26781	2
	Veggie	The Five Cheese Pizza	26066	3

THANK YOU

Pizza Sales Analysis — MySQL Query Portfolio

github.com/vikram-kaushik | linkedin.com/in/vikramkaushik007